

1. Claims → JWT Payload

```
var claims = new[]  
{  
    new Claim(ClaimTypes.Name, user.UserName),  
    new Claim(ClaimTypes.Role, user.Role)  
};
```

What this represents

This code defines the **Payload** section of the JWT.

Purpose

- Carries **user identity and authorization data**
- Used by the API to make **authorization decisions**
- Readable after Base64 decoding (not encrypted)

JWT Structure Mapping

JWT = Header.Payload.Signature

↑

Claims

Example decoded payload

```
{  
  "unique_name": "john.doe",  
  "role": "Admin",  
  "exp": 1734190000,  
  "iss": "yourIssuer",  
  "aud": "yourAudience"  
}
```

Key Points

- Claims describe **who the user is**

- Anyone can read them (Base64 encoded, not encrypted)
 - They must **not contain sensitive data** (passwords, secrets)
-

2. Key → Cryptographic Secret

```
var key = new SymmetricSecurityKey(  
    Encoding.UTF8.GetBytes(configuration["Jwt:Key"]));
```

What this represents

This is the **secret key** used for signing and validating the token.

Purpose

- Ensures **token integrity**
- Prevents **token tampering**
- Used by both:
 - Token issuer (to sign)
 - API (to validate)

JWT Structure Mapping

Header + Payload + Secret Key



Signature

Key Characteristics

- Stored securely (appsettings, Key Vault, environment variable)
 - **Never sent to the client**
 - Same key must be used for validation
-

3. Signature → JWT Signature

```
var token = new JwtSecurityToken(  
    issuer: configuration["Jwt:Issuer"],
```

```
audience: configuration["Jwt:Audience"],
claims: claims,
expires: DateTime.UtcNow.AddMinutes(60),
signingCredentials: new SigningCredentials(
    key, SecurityAlgorithms.HmacSha256)
);
```

What this represents

This creates the **Signature** part of the JWT.

How it works

1. JWT Header is created:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Header + Payload are Base64 encoded

3. They are hashed using:

```
HMACSHA256(
    Base64(Header) + "." + Base64(Payload),
    SecretKey
)
```

4. Result becomes the **Signature**

JWT Structure Mapping

Header.Payload.Signature

↑

Generated using Key

4. Complete JWT Mapping to Your Code

JWT Part	Code Section	Responsibility
Header	SecurityAlgorithms.HmacSha256	Defines algorithm
Payload	claims, issuer, audience, expires	User + metadata
Key	SymmetricSecurityKey	Secret for signing
Signature	SigningCredentials	Prevents tampering

5. Why All Three Are Required

Component Without It

Claims	No user identity
Key	Token can be forged
Signature	Token can be modified

6. Real-World Analogy

JWT Component Analogy

Claims	Passport details
Key	Government seal
Signature	Stamp verifying authenticity

7. Key Takeaway

- **Claims = WHAT** is inside the token
- **Key = SECRET** used to protect the token
- **Signature = PROOF** the token was not altered